

CZ1106
CE1106

Chapter 2

Data Organisation in Memory

©2020 SCSE/NTU

1

CZ1106
CE1106

Chapter 2

Data Organisation in Memory

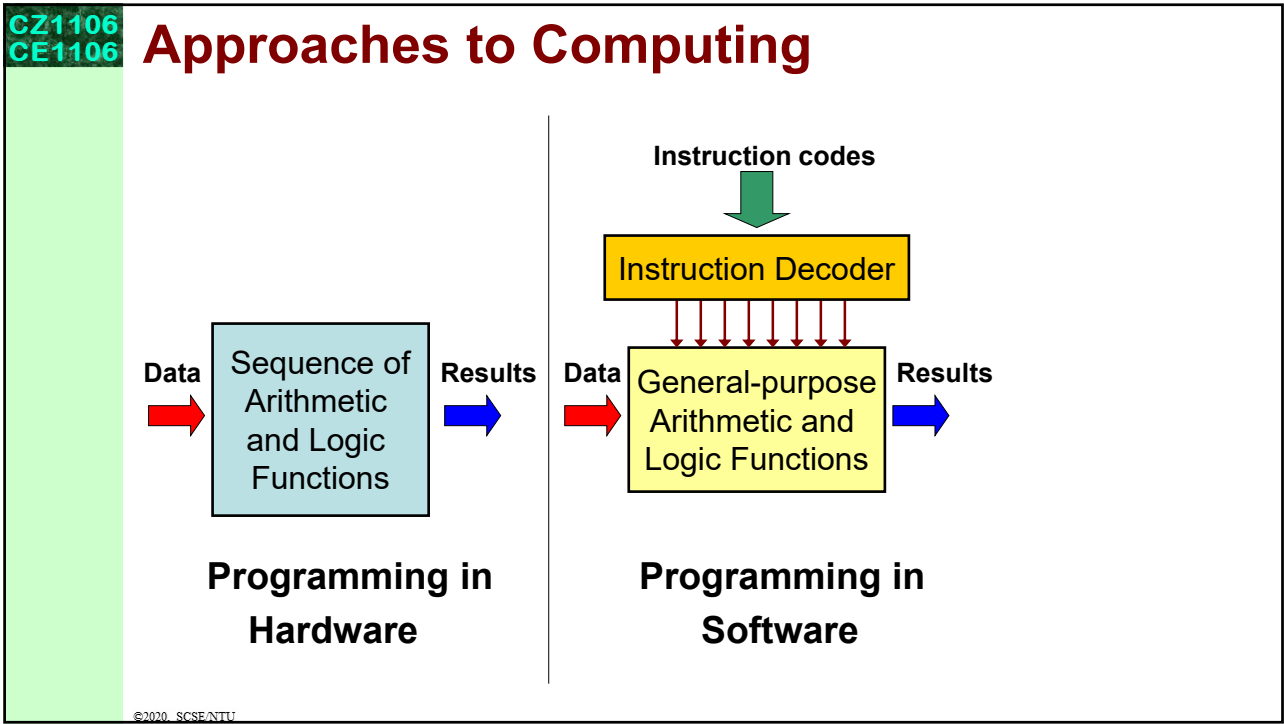
Role of Memory in Computing

Learning Objectives (2.1)

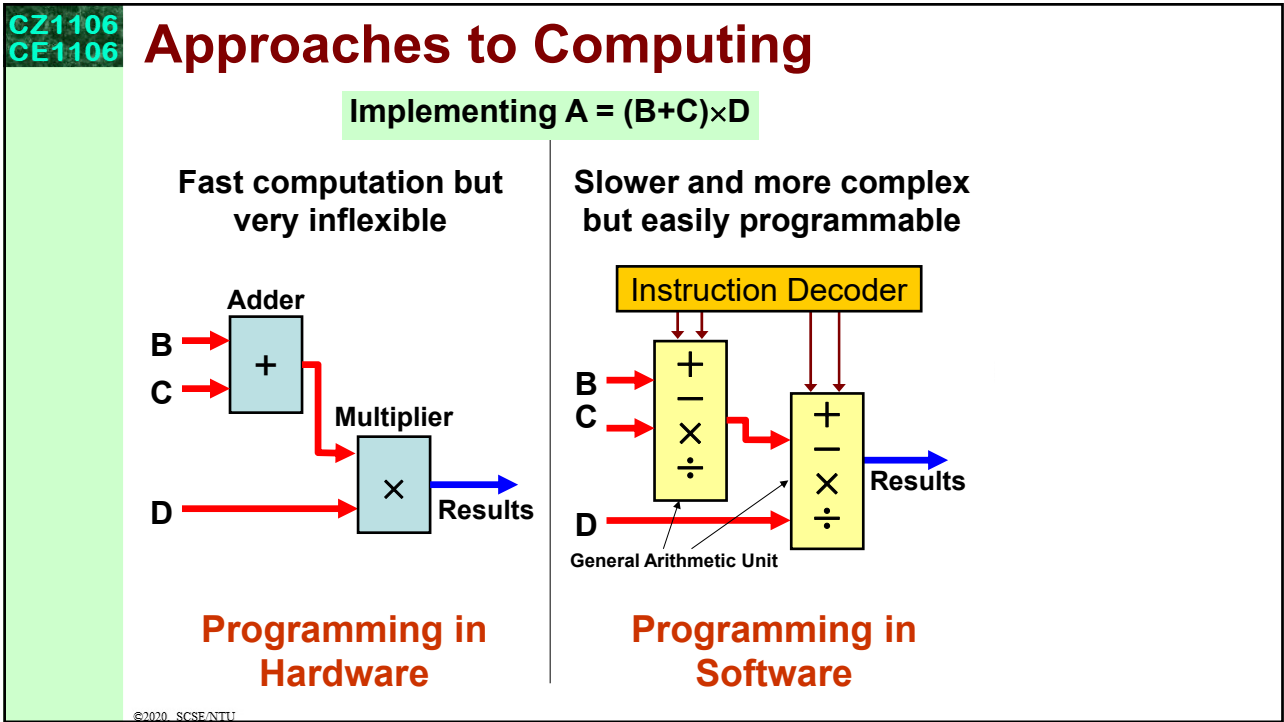
1. Describe the concept of programming in software.
2. Describe the von Neumann's stored program concept.
3. Describe the role of memory in computing.
4. Describe the characteristics and function of different data storage elements in the memory hierarchy.

©2020 SCSE/NTU

2



3



CZ1106
CE1106

Code, Data and Memory

- What is code and what is data?
 - Code** is a sequence of instructions.
 - Data** are values these instructions operate on.
- What is the memory?
 - Its a sequential list of addressable storage elements for storing both instructions and data.

e.g. B+C

Address	Content	
0x000	+	Instruction
0x001		
0x002		
:	:	
0x100	B	Data
0x101	C	
:	:	

Memory

©2020 SCSE/NTU

5

CZ1106
CE1106

The Stored Program Concept

```
graph LR; Input[Input] --> CPU[Central Processing Unit CPU]; CPU --> Output[Output]; CPU <--> Memory[Memory stores data and instructions];
```

- Most modern day computer design are based on von Neumann’s stored program concept:
 - Both data & instructions are stored in the same memory
 - Contents of memory are addressable by location, without regard to data type
 - Execution occurs sequentially (unless explicitly modified)

©2020 SCSE/NTU

6

CZ1106
CE1106

Memory Hierarchy

Memories are generally organized in levels of increasing speed and cost/bit.

The diagram illustrates the memory hierarchy from left to right: Processor (containing Registers), Cache Memory, Main Memory, and Secondary Memory. Double-headed red arrows connect Processor to Cache Memory, and Cache Memory to Main Memory. A dashed double-headed red arrow connects Main Memory to Secondary Memory. A blue arrow points from right to left below the boxes, labeled 'Increasing cost/bit and speed'.

● Registers	Very fast access but limited numbers within CPU. Operates at CPU clock rate (size: 2-128 registers)
● Cache Memory	Fast access static RAM close to CPU. Typical access time 3-20nS (size: up to 512 kB)
● Main memory	Usually dynamic RAM or ROM (for program storage). Typical access time 30-70nS. (size: up to 16GB)
● Secondary Memory	Not always random access but non-volatile. Maybe be based on magnetic or flash technology. Typical access time 0.03-100mS. (size: up to 4TB)

©2020 SCSE/NTU

7

CZ1106
CE1106

Characteristics of Main Memory

- **Fix-sized** (typically 8-bit) storage location accessible at high speed and in **any order**.
- Each byte-sized location has a unique **address** that is accessed by specifying its binary pattern on the **address bus**.
- Memory size is dependent on number of lines (n) in the address bus. (Memory size = 2^n bytes).
- Memory stores both **data** and **instructions**. **Consecutive** locations used to store multi-byte data.

The diagram shows a vertical table with two columns: 'Address' and 'Memory'. The 'Address' column contains values 0000, 0001, 0002, followed by three vertical dots, and then 2^n-1 . The 'Memory' column contains vertical dots in the first three rows and then empty boxes for the remaining rows. A horizontal double-headed arrow at the bottom of the memory boxes is labeled '8 bits'.

©2020 SCSE/NTU

8

CZ1106
CE1106

Summary

- Programming in software provides flexibility.
 - Functionality specified by bit patterns in the instructions.
 - Instructions are stored sequentially in memory.
 - Programs are executed by fetching these instructions one at a time from memory.
- Memory stores both instructions and data.
 - Memory contents are addressable by a unique address.
 - Each unique address stores a fixed number of bits (i.e. 8 bits or a byte)

©2020 SCSE/NTU

9

CZ1106
CE1106

10

©2020 SCSE/NTU

10

CZ1106
CE1106

Chapter 2

Data Organisation in Memory

Number Representation

Learning Objectives (2.2)

1. Describe the different C numeric data types and their characteristics.
2. Describe the concept of numeric range and its implications to data size.
3. Describe how multi-byte numbers are stored in memory.

©2020 SCSE/NTU

11

CZ1106
CE1106

Data Representation in Memory

- Programming languages like C has many different data types.
 - Numbers
 - Characters
 - Boolean
 - Arrays
 - Structures
 - Pointers
- How are these variables stored in memory?
 - Knowledge of their representation in memory allows us to find efficient ways to access and process them in our program.
 - Note: Lecture notes will use ANSI C programming language as an example.

©2020 SCSE/NTU

12

CZ1106
CE1106

Number Representation

- ANSI C data types representing numbers come in various varieties (basic type, sign & size).
- There are two **basic types**. Whole number or **integer** and **floating point** number.
 - Integer, declared as **int** (e.g. 32,676)
 - Floating point, declared as **float** (e.g. 3.2676×10^4)
 - Floating point numbers are useful for scientific calculations and has issue of trading off **precision** and **range** for a given size (in bits).
(to be covered in later lectures in Computer Arithmetic)
 - Some CPUs are only integer-based and make use of an additional floating point unit (FPU) to support floating point computations.

©2020 SCSE/NTU

13

CZ1106
CE1106

Number Representation (cont)

- Floating point numbers are always **signed**.
- Integers can be either **signed** or **unsigned**.
 - Signed integer, declared as **int** (e.g. -1)
 - Unsigned integer, declared as **unsigned int** (e.g. 255)
 - Most processors interpret signed numbers using the **2's complement** representation.

2's complement	Unsigned
0111 1111 ₂ = (127)	0111 1111 ₂ = (127)
1111 1111 ₂ = (-1)	1111 1111 ₂ = (255)

- Use unsigned numbers where possible to increase the positive range (e.g. counting a population).

Learn More: Google "Signed number representation"

©2020 SCSE/NTU

14

CZ1106
CE1106

Number Representation (cont)

- The **range** can be increased by using more bytes to represent the number.
- Use appropriate suffix (**short** and **long**) to specify required size.
- Use appropriate size to reduce memory requirements of your program.

Type	Bytes	Bits	Range
signed char	1	8	-128 -> +127
unsigned char	1	8	0 -> +255
short int	2	16	-32,768 -> +32,767 (+/-32KB)
unsigned short int	2	16	0 -> +65,535 (64KB)
unsigned int	4	32	0 -> +4,294,967,295 (4GB)
int	4	32	-2,147,483,648 -> +2,147,483,647 (+/-2GB)
long int	4	32	-2,147,483,648 -> +2,147,483,647 (+/-2GB)
long long int	8	64	-(2^63) -> (2^63)-1
float	4	32	
double	8	64	
long double	12	96	

ANSI C numeric data types and their respective size and range

Note: These sizes may change depending of the processor and compiler

©2020 SCSE/NTU

15

CZ1106
CE1106

Data Organization in Memory

- How is a 32-bit number stored in memory?
- There are two ways, depending on the **byte-ordering** of the data in memory

Big Endian format

Address | Memory

N
N+1
N+2
N+3

8 bits

Address of 32-bit word

(e.g. Mac PowerPC, Sun SPARC, Motorola 68k)

Little Endian format

Memory | Address

N
N+1
N+2
N+3

Address of 32-bit word

(e.g. Intel x86, DEC Alpha, 8051)

32-bit data

Most significant byte

Least significant byte

©2020 SCSE/NTU

Learn More: Google "Byte ordering"

16

CZ1106
CE1106

Summary

- Numbers are represented in a variety of ways.
 - Number of bits.
 - Integer or Floating Point.
 - Signed or Unsigned.
- For a given data size (i.e. number of bits), range and precision must trade off against each other.
- Multi-byte integers are stored using either Big or Little Endian byte-ordering.

©2020 SCSE/NTU

17

CZ1106
CE1106

18

©2020 SCSE/NTU

18

CZ1106
CE1106

Chapter 2

Data Organisation in Memory

Character and Boolean Representation

Learning Objectives (2.3)

1. Describe the char data type and its representations.
2. Describe the Boolean data type and its implementation.

©2020 SCSE/NTU

19

CZ1106
CE1106

Character Representation

- Computer not only process numbers but handles character data (e.g. text processing, print display).
- In ANSI C, a character type is declared as **char**.
- A char variable required **one byte** of memory storage
- Data in a computer is stored in **binary** but they are transformed into representative characters through some **encoding standard**.
- The most common character encoding standard is the 7-bit **ASCII** code.
- There are many other character encoding standards - DEC's Sixbit (6-bit), IBM's EBCDIC (8-bit) and Unicode (16-bit), etc.

```
main()
{
    char c;    //declare a character variable

    c = 'a';  //assign character 'a' to variable
}
```

Learn More: Google "Character Codes"

©2020 SCSE/NTU

20

CZ1106
CE1106

ASCII

- American Standard Code for Information Interchange (ASCII) is a 7-bit code for representing characters.
- Useful properties – lower, upper and digits are **contiguous**, which makes it easy to check character's category and also transpose it from one case to another.
- A byte is normally used to store an ASCII character and MSB could be used for **parity error checking**.

MS \ LS	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P	'	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	w	w
8	BS	CAN	(	8	H	X	x	x
9	HT	EM	)	9	I	Y	y	y
A	LF	SUB	*	:	J	Z	z	z
B	VT	ESC	+	;	K	[	{	{
C	FF	FS	,	<	L	\		
D	CR	GS	-	=	M	]	~	~
E	SO	RS	.	>	N	^	o	o
F	SI	US	/	?	O	_		DEL

Bit 7

1100001

Bit 0

0x41 = "A"

Odd Parity

7-bit ASCII Table

Learn More: Google "Parity bit and ASCII"

21

CZ1106
CE1106

SIXBIT

- DEC's SIXBIT character code was popular in the 1960's and 70's.
- Not used for general text processing as it lacks control characters like CR and LF.
- Six-bit character format was popular with DEC's PDP-8 and PDP-10 computers, which used 18-bit and 36-bit processors respectively.

MS \ LS	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
1	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
2	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
3	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_

SIXBIT Table

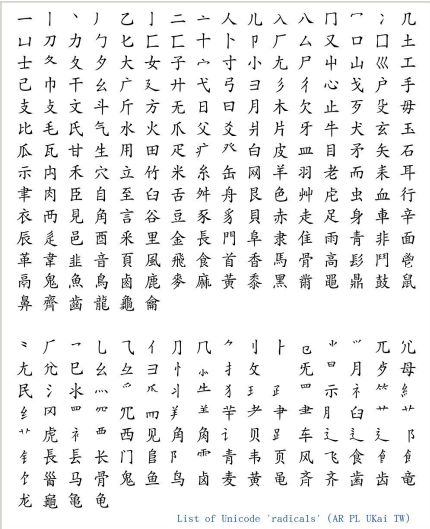
22

CZ1106
CE1106

Unicode

- Developed to handle text expressions for all major living languages in the world.
- An **8,16 or 32-bit** character encoding standard that is downward compatible with ASCII.
- Adopted by technologies such as XML, Java programming language, Microsoft .NET Framework and many operating systems.
- Unicode 13.0 was launched on 20 March 2020. It contains a total of 143,859 characters.

Unicode radicals for KangXi
214 radicals to support Chinese, Japanese and Korean ideographs (U+2F00 – U+2FDF)



List of Unicode 'radicals' (AR PL UKai TW)



Learn More: Google "Unicode" or "Unicode characters"

©2020 SCSE/NTU

23

CZ1106
CE1106

Boolean Representation

- Boolean variables have only 2 states.
- The Boolean type was made available in ANSI C (after 1999) as `_Bool` with the `stdbool.h` header file.
- Values assigned in C: **False** = 0, **True** = non-zero (e.g. 1)
- Memory storage for Boolean variables is **inefficient** as most implementation use a byte (minimum memory unit) to store a 1-bit Boolean value.
- The 8051 processor has 128 bit-addressable memory locations.

Address	Memory	
0000	0x00	false
0001		
0002	0x01	true
:	:	
:	:	
:	:	
2 ⁿ -1		

← 8 bits →

©2020 SCSE/NTU

24

CZ1106
CE1106

8051 - Bit Addressable Memory

- The 8051 processor support a small portion of bit-addressable memory.
- In embedded applications, data variables are often related to ON-OFF status of discrete sensors and output (e.g. switches or LEDs)

Lower 128 Bytes of Internal RAM 8051

8051 8-bit Processor

©2020_SCSE/NTU

www.CircuitsToday.com

25

CZ1106
CE1106

Summary

- Characters data type is used to handle text.
- They are byte-sized.
- They need an encoding standard (ASCII being the most common).
- In the C programming language they are declared using **char**.
- Boolean data types are used to represent true and false states.
- They are usually stored using a whole byte.
- In ANSI C, the **false** value is given by the numeric value 0.
- The **true** value is taken as any non-zero value.

©2020_SCSE/NTU

26

CZ1106
CE1106

Chapter 2

Data Organisation in Memory

Array, String and Structure Representations

Learning Objectives (2.4)

1. Describe the representation of arrays in memory.

2. Describe the C and Pascal strings storage in memory.

3. Describe the representation of structures in memory

©2020, SCSE/NTU

27

CZ1106
CE1106

Array Representation

- A linear array is a consecutive area in memory storing a series of homogenous data type.
- Elements of an array are accessed through appropriate offset from the **base address** (BA) of the array.

```
main()
{
    char c[2];    //2 element char array c
    c[0]='A';    //assign "A" to 1st element
    c[1]='B';    //assign "B" to 2nd element
}
```

Offset from base address (BA_c) to access each element of the array c

BA_c

BA_c+1

Address

Memory

0x100

'A'

$c[0]$

0x101

'B'

$c[1]$

0x102

0x103

0x104

0x105

0x106

0x107

0x108

← 8 bits →

©2020, SCSE/NTU

28

CZ1106
CE1106

Array Representation (cont)

- Computation of the address of array element must take into account its data type size.
- Address is computed by multiplying element number by size of data type before addition to the base address (BA).

```
main()
{
    char c[2];           //2 element char array c
    short int i[3];      //3 element integer array i
    i[0] = 5;           //assign values to array i
    i[1] = 6;
    i[2] = 7;
}
```

	Address	Memory	
BA_c	0x100		c[0]
BA_c+1	0x101		c[1]
BA_i	0x102	0x00	i[0]
	0x103	0x05	
BA_i+2	0x104	0x00	i[1]
	0x105	0x06	
BA_i+4	0x106	0x00	i[2]
	0x107	0x07	
	0x108		

← 8 bits →

Offset ($n \times 2$) from base address (BA_i) to access each element n of the array i consisting of 2 byte-sized short integers

29

CZ1106
CE1106

Nested Array

- Each element of an array can itself be an array.
- General principles of array allocation and referencing hold for nested array.
- An array of arrays is then created.

```
main()
{
    int k[3][2]; //a 3x2 integer array
}
```

Offset from BA	Address	Element in Memory
BA_k	0x0100	k[0][0]
BA_k+4	0x0104	k[0][1]
BA_k+8	0x0108	k[1][0]
BA_k+12	0x010C	k[1][1]
BA_k+16	0x0110	k[2][0]
BA_k+20	0x0114	k[2][1]
	0x0118	:

← 32 bits →

$4 * ((2 * 2) + 1) = 20$

The offset from BA for element $k[a][b]$
= $\text{sizeof}(\text{int}) * ((2 * a) + b)$

30

CZ1106
CE1106

String Representation

- A string in a C program is an array of characters terminated by a null (**0x00**) character.

```
main()  
{  
    char s[4]="123"; //a string constant  
}
```

Base Address	Address	Content in Memory
BA _s	0x0100	0x31
	0x0101	0x32
	0x0102	0x33
	0x0103	0x00
	0x0104	:
	0x0105	:
	0x0106	:

← 8 bits →

C string

Address	Content in Memory
0x0100	0x03
0x0101	0x31
0x0102	0x32
0x0103	0x33
0x0104	:
0x0105	:
0x0106	:

← 8 bits →

Pascal string

Which method is better?

Learn more: Google Search "C strings vs Pascal strings"

31

CZ1106
CE1106

Structure Representation

- Structure allows new data types to be created by combining objects of different types.
- Each data type in a **declared** structure variable occupies predefined consecutive locations based on data type size.

```
main()  
{  
    struct Man ; //define structure Man  
    {  
        char race;  
        int age;  
        char name[20];  
    };  
    Man m;  
}
```

Offset from BA	Address	Memory	
BA _m	0x100		m.race
BA _m +1	0x101		
	0x102		
	0x103		
BA _m +5	0x104		m.age
	0x105		
	0x106		m.name
	:	:	
	0x118		

← 8 bits →

©2020 SCSE/NTU

32

CZ1106
CE1106

Summary

- Arrays stores a series of similar data types in consecutive memory locations.
 - Elements are accessed using offsets from a base address.
 - Offset computation must take into account size of data type.
- Structures stores a series of dissimilar data types in consecutive memory locations.
 - Memory space for structure data type is only allocated when structure variables are declared.

©2020 SCSE/NTU

33

CZ1106
CE1106

34

©2020 SCSE/NTU

34

CZ1106
CE1106

Chapter 2

Data Organisation in Memory

Pointer Representation

Learning Objectives (2.5)

1. Describe the representation of pointers in memory.

©2020 SCSE/NTU

35

CZ1106
CE1106

Pointers Representation

- Pointers in C provides a mechanism for referencing memory variables, elements of structures and arrays.
- C pointers are declared to point to a particular data type.
- The value of a pointer is an **address**. Its **size** is **fixed** (regardless of data type).
- The size of the pointer depends on the processor's address range.

```
main()
{
    char c ;           //char variable c
    char *ptr;         //char pointer ptr
    c = 'A' ;          //assign value 'A' to c
    ptr = &c;          //ptr gets address of c
}
```

Address	Content in Memory
0x0100	'A'
0x0101	:
0x0102	:
0x0103	:

ptr → 0x0100 'A' → c

← 16 bits → ← 8 bits →

Variable	Size (Bytes)
c	1
ptr	2

Note: 1. Assume address of c = 0x0100
2. Assume addresses in CPU are specified using 16 bits (2 bytes).

©2020 SCSE/NTU

36

CZ1106
CE1106

Dereferencing a Pointer

- When properly initialized, a pointer contains the start address where the memory variable can be found.
- We can use the dereferencing operator (*) to copy a value to the address pointed to by the pointer **ptr**.
- Knowing the start address (base address) of an array or structure makes it easy to access their different elements.

```
main()
{
    char c ;           //char variable c
    char *ptr;         //char pointer ptr
    c = 'A' ;          //assign value 'A' to c
    ptr = &c;          //ptr gets address of c
    *ptr = 'Z' ;        //use dereferencing
                        //operator on ptr to give
                        //variable c value 'Z'
```

Address	Content in Memory
0x0100	'Z'
0x0101	:
0x0102	:
0x0103	:

ptr → 0x0100 'Z' → c

← 16 bits → ← 8 bits →

©2020 SCSE/NTU

37

CZ1106
CE1106

Summary

- Pointer is a variable whose value is an address.
- The size of a pointer variable is independent of data type it is pointing to.
- The size of a pointer is dependent on the addressable memory space of the processor.

©2020 SCSE/NTU

38

CZ1106
CE1106

Chapter 2

Data Organisation in Memory

Data Alignment

Learning Objectives (2.6)

1. Describe the concept of data alignment.

2. Describe data alignment considerations for efficient access and storage of structure variables in memory.

©2020 SCSE/NTU

39

CZ1106
CE1106

Data Alignment

• Most computer systems have restrictions on the allowable address for accessing various data types.

• Multi-byte data (e.g. **int**, **double**) must be aligned to addresses that are multiple of values such as 2, 4, or 8.

• Programs written with Microsoft (Visual C++) or GNU (gcc) and compiled for a 64-bit Intel processor will use the following data alignment enforcement:

Data Type	Size (Byte)	Example of allowable start addresses due to alignment
char	1	0x..0000, 0x..0001, 0x..0002
short	2	0x..0000, 0x..0002, 0x..0004
int	4	0x..0000, 0x..0004, 0x..0008
float	4	0x..0000, 0x..0004, 0x..0008
double	8	0x..0000, 0x..0008, 0x..0010
pointer	8	0x..0000, 0x..0008, 0x..0010

©2020 SCSE/NTU

40

CZ1106

CE1106

Memory Interfacing & Data Alignment

(Case Study: Intel Pentium 4)

- Main memory consist of multiple 8-bit memory modules required to make up 64-bit data word size of processor.
- Data width is selected by additional control lines (BE0-BE7).

- Only one address pattern can exist on address bus during data transfer. 8-byte data needs **two memory cycles** if not aligned to 8-byte address boundary.

Learn More: Google "memory and data alignment"

©2020 SCSE/NTU

41

CZ1106

CE1106

Address Mapping of 64-bit data

(Case Study: Intel Pentium 4)

Address Pins	A ₃₅	A ₃₄		A ₀₆	A ₀₅	A ₀₄	A ₀₃	A ₀₂	A ₀₁	A ₀₀
Byte 1	0	0		0	0	0	0	1	0	0
Byte 2	0	0		0	0	0	0	1	0	1
Byte 3	0	0		0	0	0	0	1	1	0
Byte 4	0	0		0	0	0	0	1	1	1
Byte 5	0	0		0	0	0	1	0	0	0
Byte 6	0	0		0	0	0	1	0	0	1
Byte 7	0	0		0	0	0	1	0	1	0
Byte 8	0	0		0	0	0	1	0	1	1

Note: Address pin A₀₃ cannot be in states 0 and 1 simultaneously to transfer a mis-aligned word in a single memory cycle.

Note: Lines BE0-BE7 implements address bits A₀₂ to A₀₀ and determine which byte(s) in the 8 possible addresses are accessed during memory transfer.

42

CZ1106
CE1106

Data Alignment and Data Bus Width

(Case Study: An 8-bit Processor - Intel 8080)

- The extent of data alignment is dependent on the width of the processor's data bus.
- A processor with an **8-bit** data bus does not have any data alignment issues. Multi-byte data can be fetched from **any address**.
- A processor with a **16-bit** (i.e. 2 bytes) data bus only needs to align 2, 4 or 8 byte-sized data to **even addresses** (e.g. 0x0000, 0x0002, 0x0004, etc).

Address	Memory
0x0..000	2-byte data (Byte 1)
0x0..001	Byte 2
0x0..002	2-byte data (Byte 3)
0x0..003	Byte 4
0x0..004	
0x0..005	
0x0..006	

← 8 bits →

Note: It takes 2 transfer cycles to read Bytes 1 and 2
It also takes 2 transfer cycles to read Bytes 3 and 4

©2020 SCSE/NTU

43

CZ1106
CE1106

Data Alignment with Structures

- Data padding** (addition of meaningless bytes) is used by compilers to ensure alignment of different data types within a structure.

```
struct rec1 {  
    char c;  
    int i;  
    char d[2];  
}  
rec1 r;
```

No Data Padding

Address	Memory
0x000	r.c
0x001	
0x002	r.i
0x003	
0x004	
0x005	r.d[0]
0x006	r.d[1]
0x007	
0x008	
0x009	
0x00A	
0x00B	

Data alignment violation

Data Padding

Address	Memory
0x000	r.c
0x001	
0x002	
0x003	
0x004	
0x005	r.i
0x006	
0x007	
0x008	r.d[0]
0x009	r.d[1]
0x00A	
0x00B	

Padded bytes

©2020 SCSE/NTU

44

CZ1106
CE1106

Data Alignment with Structures

- **Data padding** (addition of meaningless bytes) is used by compilers to ensure alignment of different data types within a structure.
- Padded bytes are added between end of last structure element and the start of the next to maintain alignment in an array of structures.

```
struct rec1 {
    char c;
    int i;
    char d[2];
}
rec1 r;
:
rec1 t[10];
```

No Data Padding

Address	Memory
0x000	r.c
0x001	
0x002	r.i
0x003	
0x004	
0x005	r.d[0]
0x006	r.d[1]
0x007	
0x008	
0x009	
0x00A	
0x00B	

Data alignment violation

Data Padding

Address	Memory
0x000	r.c
0x001	
0x002	
0x003	
0x004	
0x005	r.i
0x006	
0x007	
0x008	r.d[0]
0x009	r.d[1]
0x00A	
0x00B	

Padded bytes

45

CZ1106
CE1106

Data Alignment with Structures (cont)

- Structures should be constructed with data alignment constraints in mind.
- **Rearrange the order** of data objects in the structure based on their size to minimize data padding where possible.

```
struct rec2 {
    int i;
    char c;
    char d[2];
}
rec2 s;
```

Data Padding

Address	Memory
0x000	
0x001	s.i
0x002	
0x003	
0x004	s.c
0x005	s.d[0]
0x006	s.d[1]
0x007	
0x008	

Total = 8 bytes

Padded byte

Before re-arranging structure (previous)

Address	Memory
0x000	r.c
0x001	
0x002	
0x003	
0x004	
0x005	r.i
0x006	
0x007	
0x008	r.d[0]
0x009	r.d[1]
0x00A	
0x00B	

Total = 12 bytes

Learn More: Google "Structure and Padding in C"

46

CZ1106
CE1106

Summary

- Data alignment restricts where in memory multi-byte data can be stored.
- Data must be aligned to addresses that are multiple of the size of their data type.
- The extent of the data alignment restriction depends on the size of the processor's data bus.
- Structure data types should consider data alignment issues.
- Data padding is used to meet alignment restrictions.

©2020 SCSE/NTU